# *Entregable final Ciencias de datos en producción: Jenkins, Vertex AI y AutoML en Google Cloud Platform*

*Mateo Monsalve Valencia*

*9 de noviembre de 2026*

## *1. Implementación de CI/CD en Jenkins*

Se implementó un flujo de integración continua y entrega continua (CI/CD) en Jenkins. El servicio se desplegó en Google Cloud Platform (GCP), utilizando Compute Engine con una instancia `e2-medium` en la región `us-central1`.




Figura 1

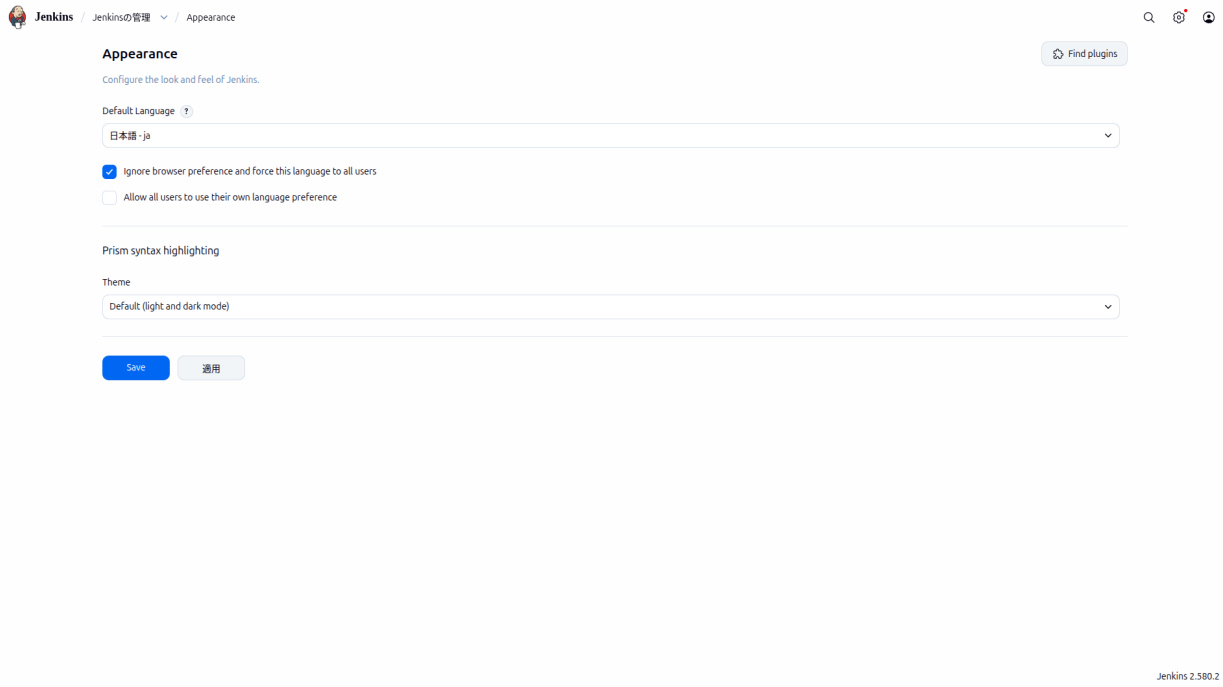



Figura 2

Se configuraron dos flujos de ejecución en Jenkins: uno para la rama `master` y otro para las demás ramas. El flujo de `master` ejecuta las pruebas, el entrenamiento, los controles de calidad, la aprobación y la notificación. En las demás ramas se ejecutan únicamente las pruebas automáticas y se omiten las etapas de entrenamiento y validación de los modelos.

## 1.1. Activación automática con webhooks de GitHub

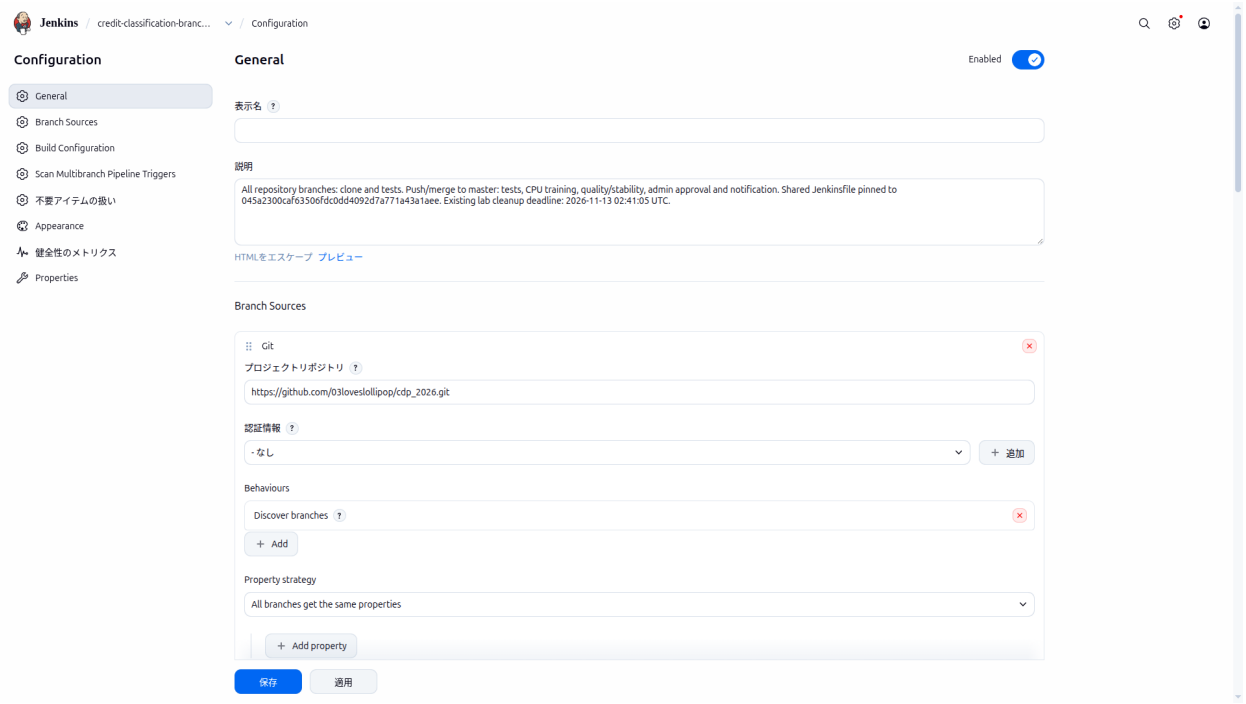



Figura 3

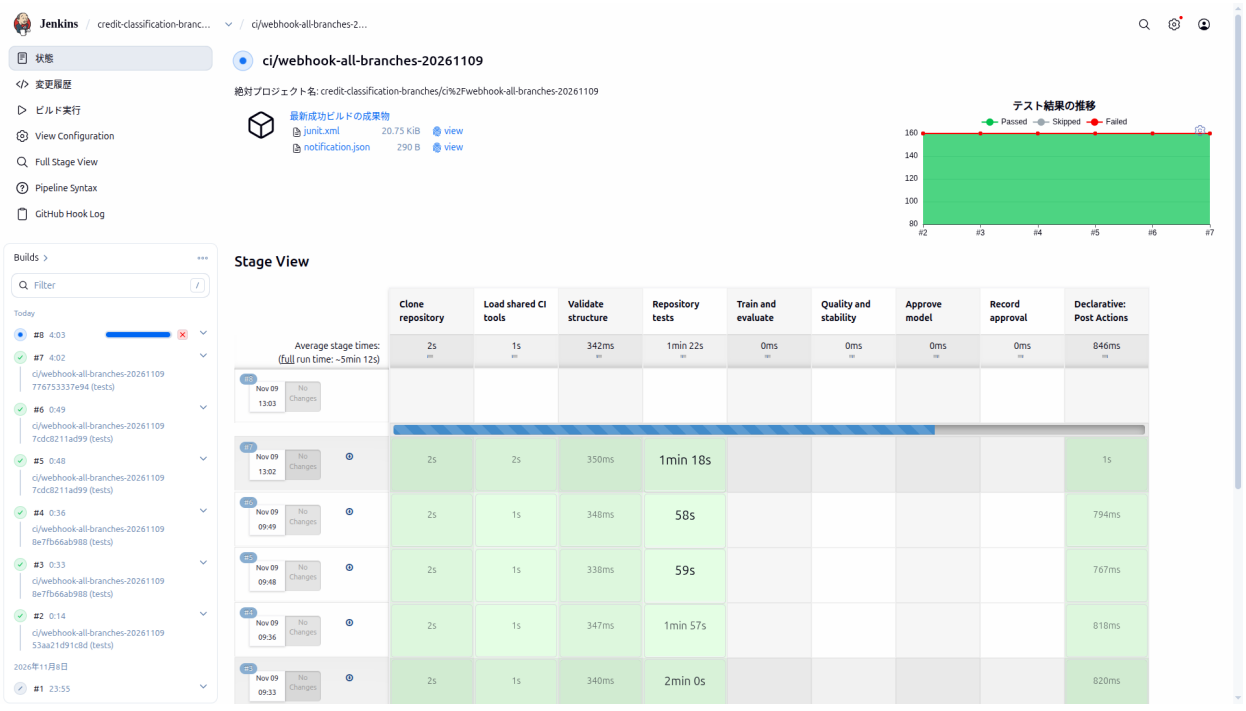



Figura 4

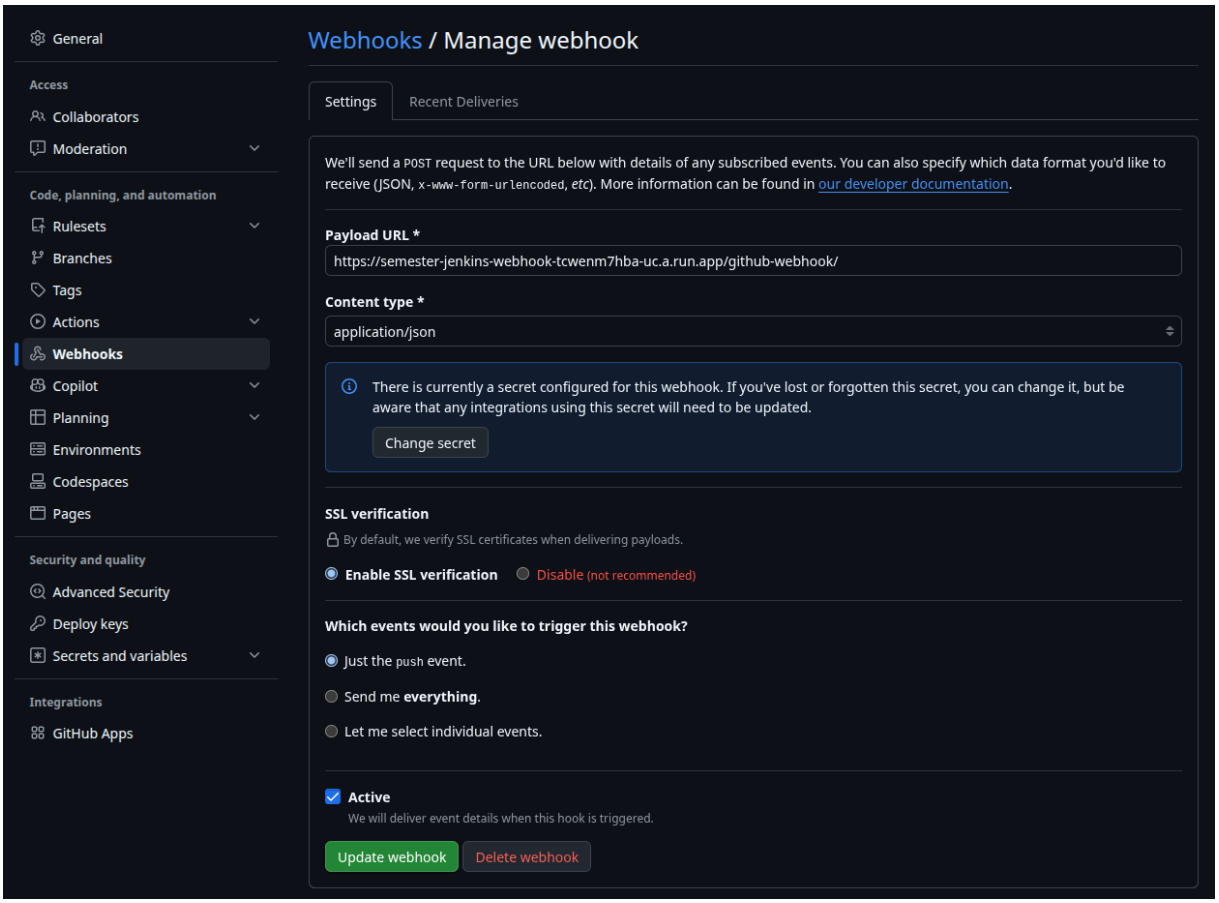



Figura 5

El merge del PR #18 actualizó la rama destino y activó el build #9, que aprobó las 160 pruebas. Jenkins registró tanto el evento de push como la indexación de la rama. Por esta razón, esa ejecución no permite atribuir el inicio únicamente al webhook ni asegurar que cada cambio se ejecute una sola vez.

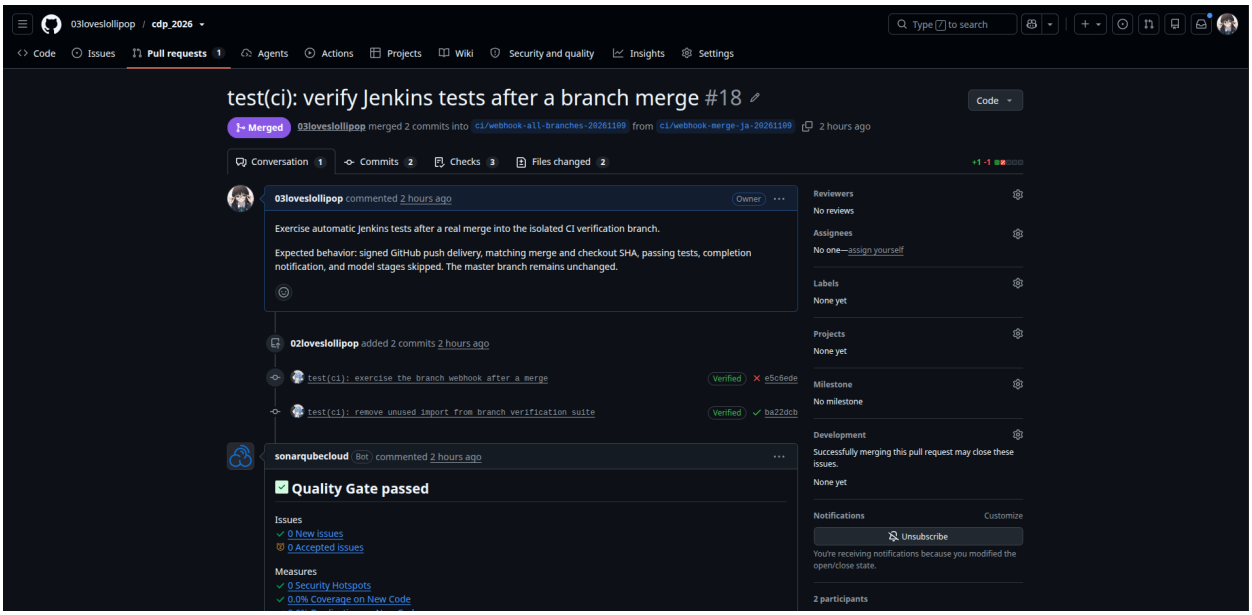



Figura 6

También se verificó la calidad del código con SonarCloud. Como se observa en la captura, el control de calidad se aprobó con cero incidencias nuevas.

### *1.2. Validación inicial de la integración con GitHub*

Antes de la validación final, se comprobó la integración de GitHub con un job de laboratorio. El push y el merge del PR #16 activaron Jenkins y el build de merge aprobó 186 pruebas. Las capturas siguientes corresponden a esa revisión inicial; sus conteos se presentan separados de las ejecuciones finales.




Figura 7

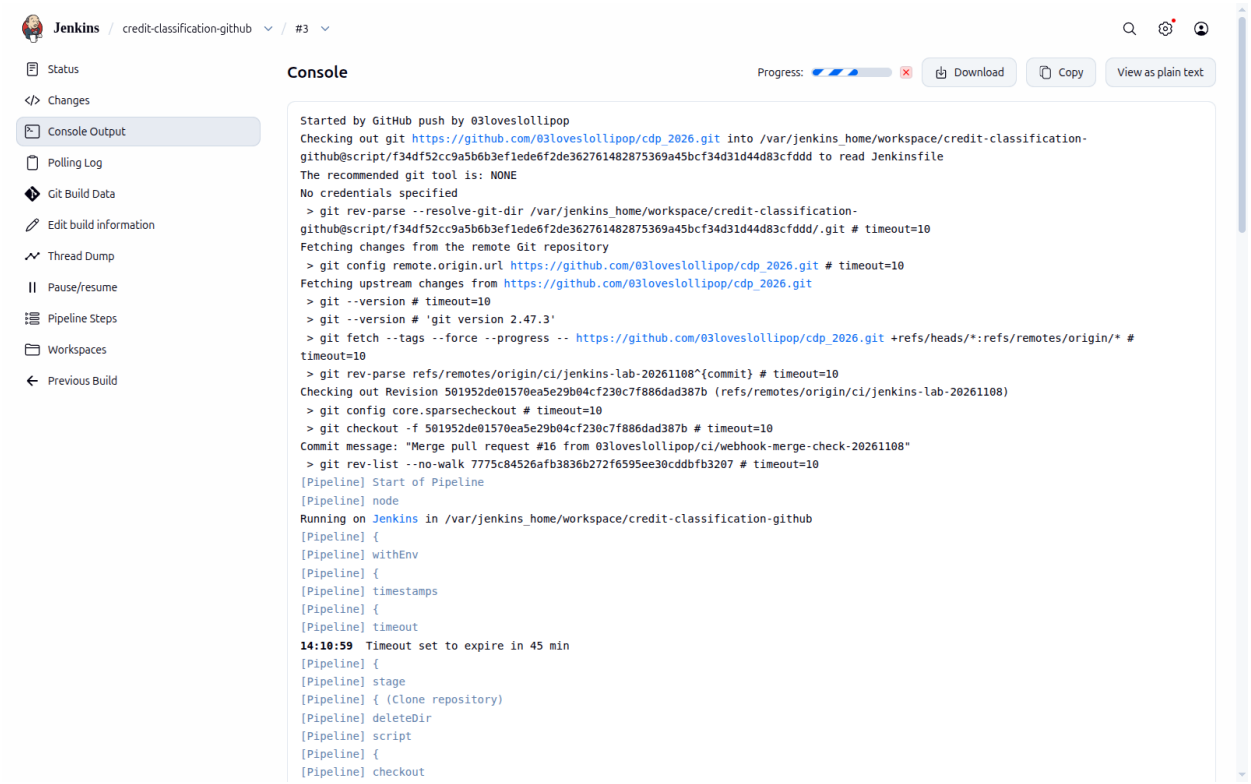



Figura 8

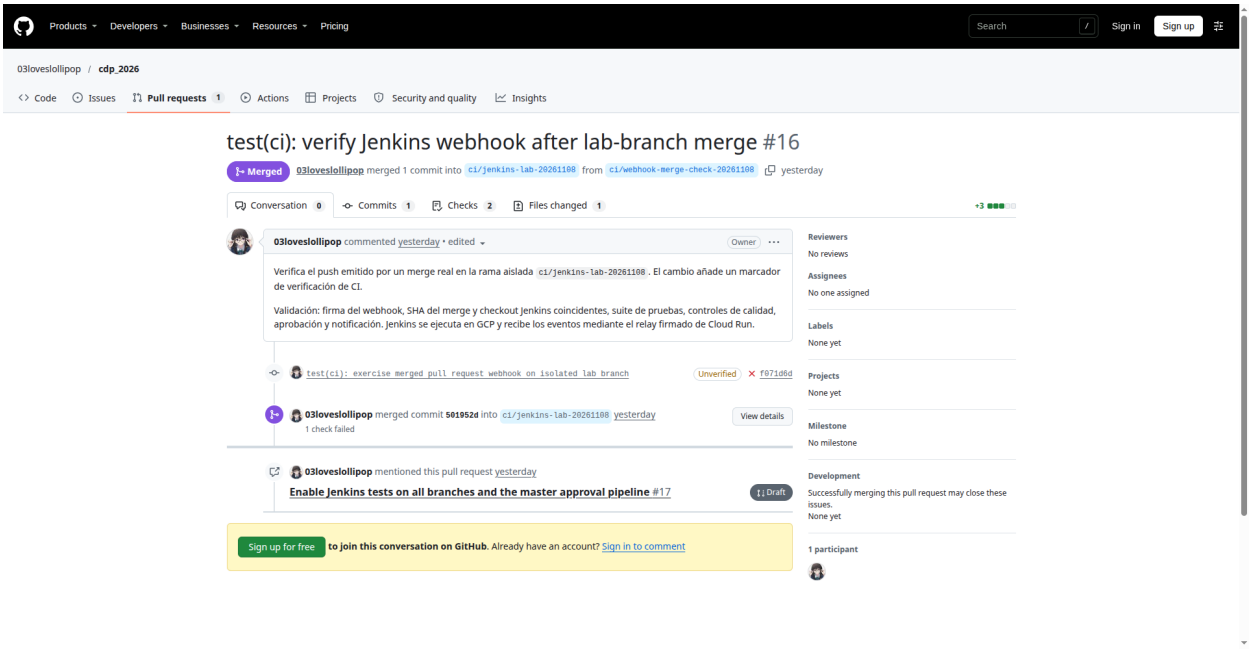



Figura 9

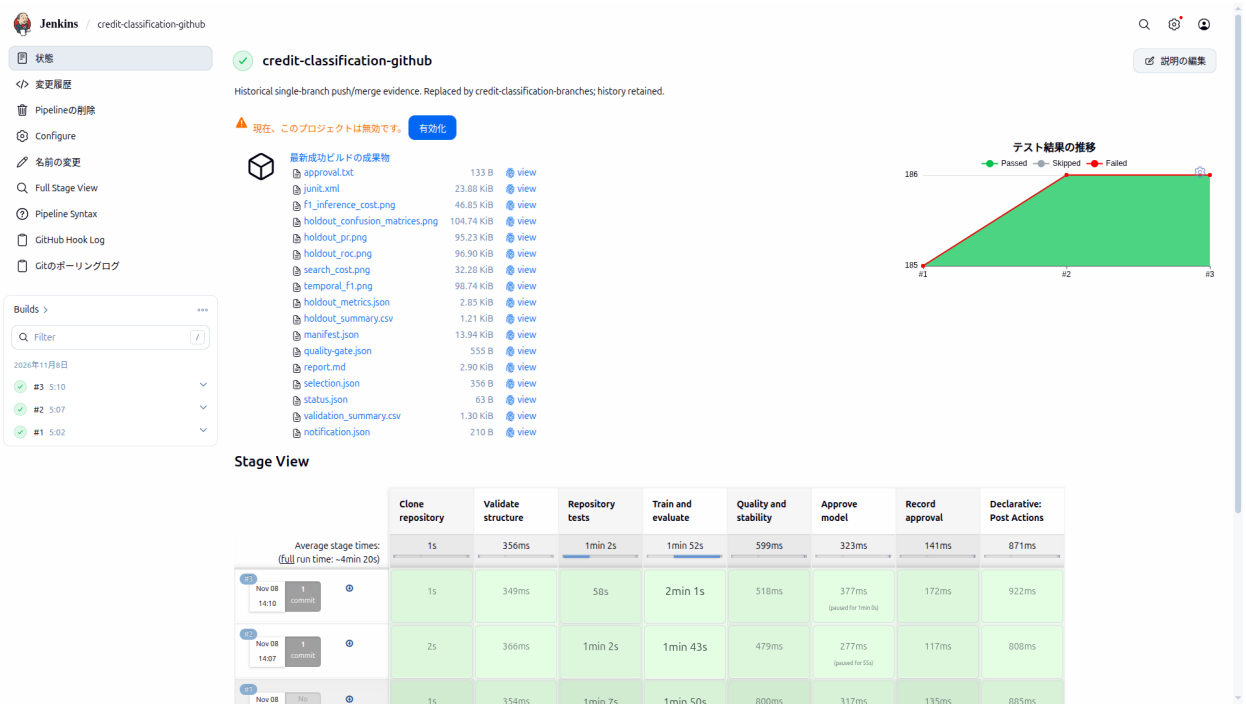



Figura 10

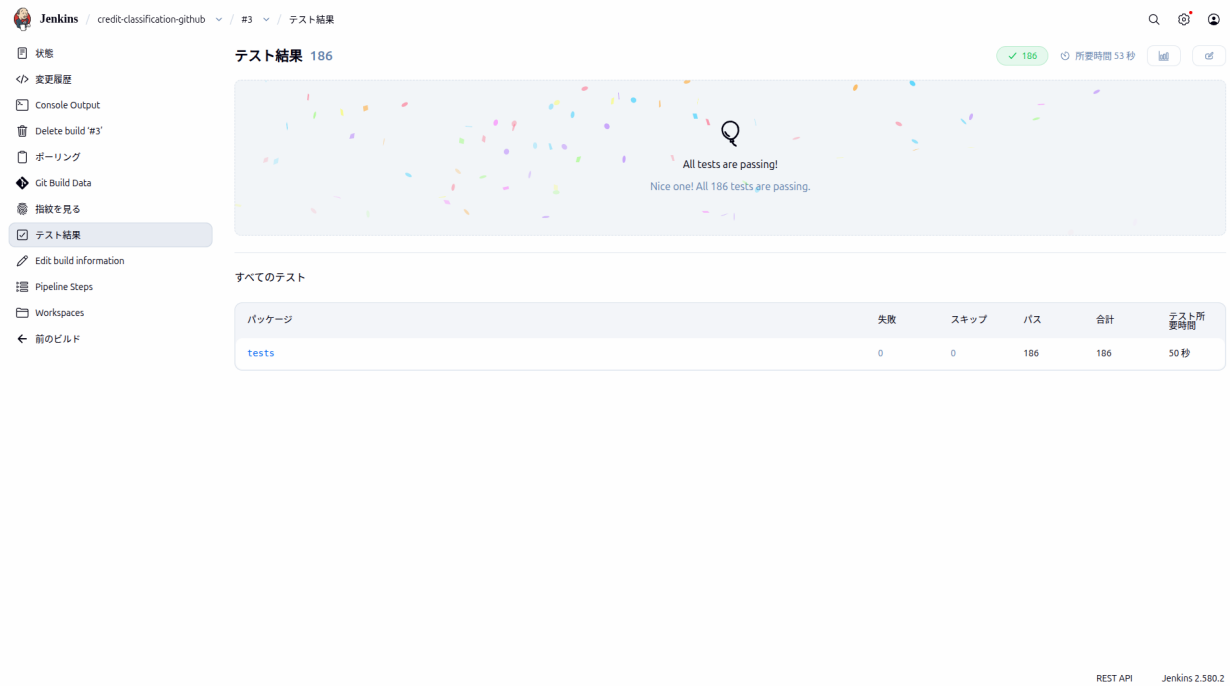



Figura 11




Figura 12

### *1.3. Resultados del control de calidad del modelo seleccionado*

Se agregó un control de calidad para el modelo seleccionado. Las métricas se evaluaron con validación temporal antes de revisar el conjunto de prueba (TEST). Los cuatro criterios definidos se cumplieron, como se muestra en la tabla.

| **Control** | **Valor medido** | **Política** | **Resultado** |
|---|---|---|---|
| F1 medio de validación | 0,1871 | $\geq$ 0,10 | Cumple |
| ROC-AUC medio de validación | 0,6650 | $\geq$ 0,50 | Cumple |
| Desviación F1 temporal | 0,0292 | $\leq$ 0,15 | Cumple |
| Desviación F1 entre semillas | 0,0110 | $\leq$ 0,05 | Cumple |

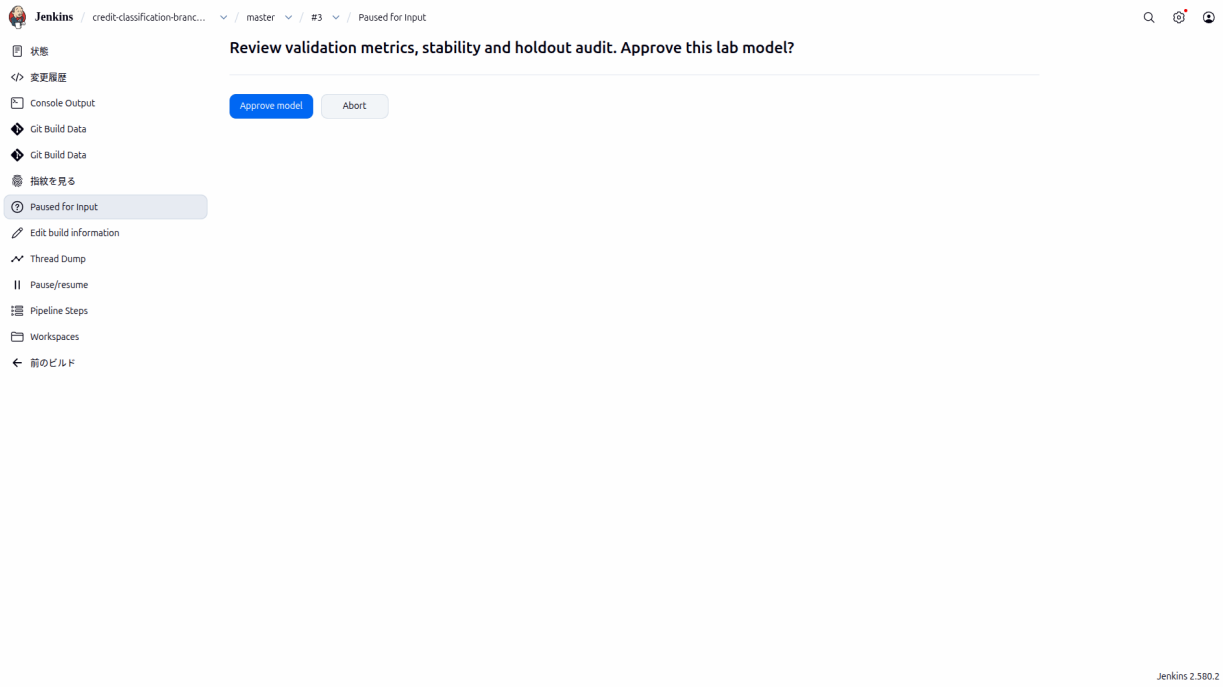



Figura 13

Antes de aceptar un cambio de modelo, se requiere la aprobación de un administrador. La captura muestra la etapa de aprobación pendiente, con el build #3 de `master` listo para revisión. Los controles de calidad se cumplieron y el build quedó en espera de la aprobación final.

La validación inicial también incluyó una pausa para la aprobación de un administrador. La siguiente captura corresponde a ese primer flujo de Jenkins.

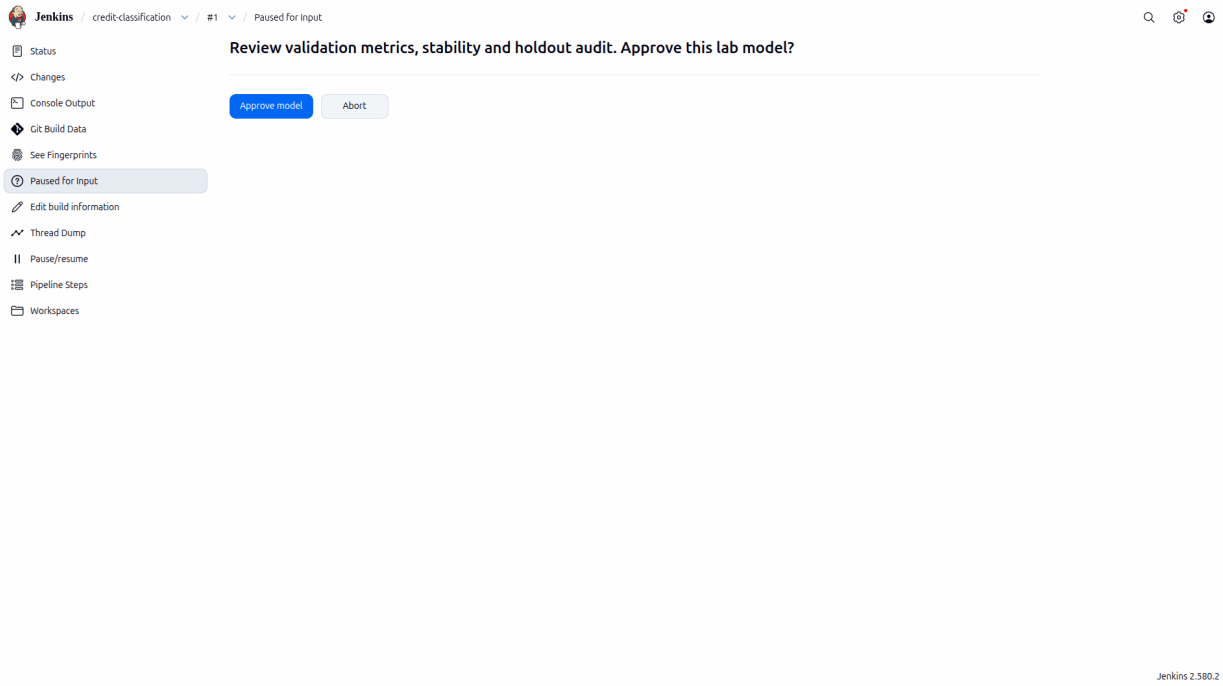



Figura 14

### *1.4. Ejecución, pruebas y notificaciones*

Durante la validación inicial se observó el pipeline en ejecución y se verificaron las pruebas y la notificación de cierre. Los builds local #5 y cloud original #1 finalizaron con 165 pruebas aprobadas. Estas ejecuciones pertenecen a una revisión distinta de la suite utilizada en las pruebas posteriores.

| Validación histórica | Resultado | Pruebas aprobadas |
|---|---|---|
| Jenkins local #5 y cloud original #1 | SUCCESS | 165 en cada entorno |
| Integración GitHub, job de laboratorio #3 | SUCCESS | 186 |

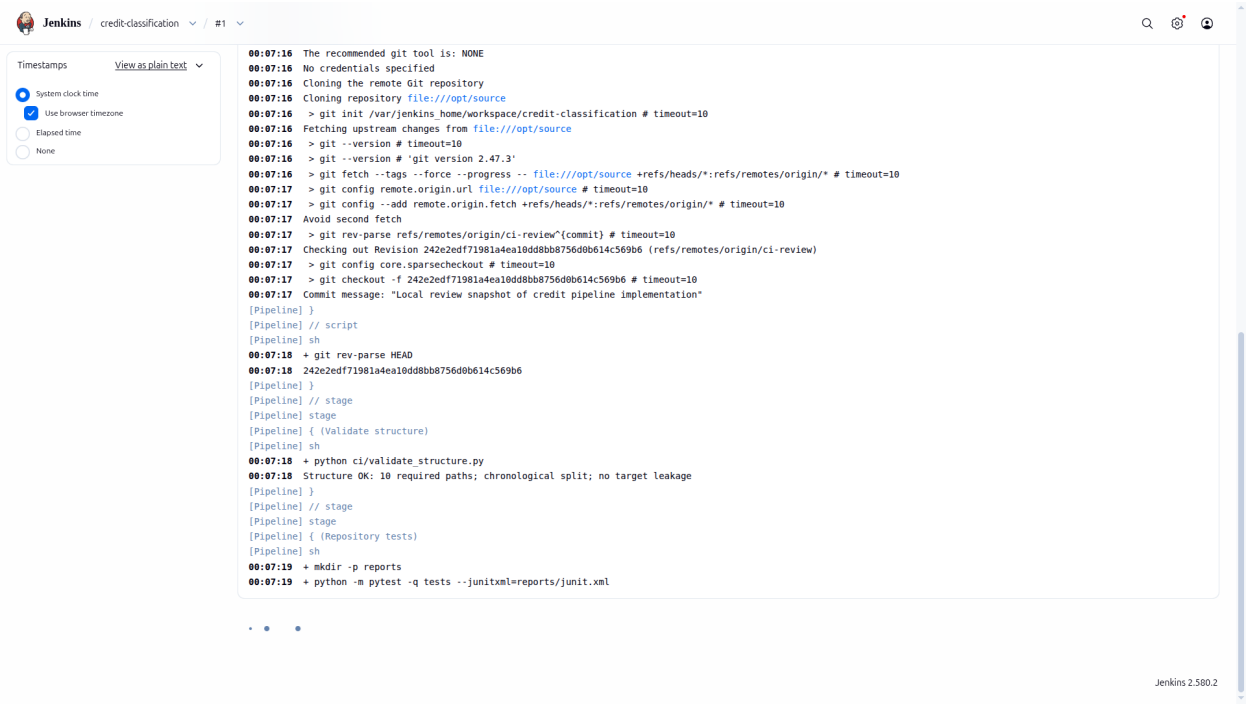



Figura 15

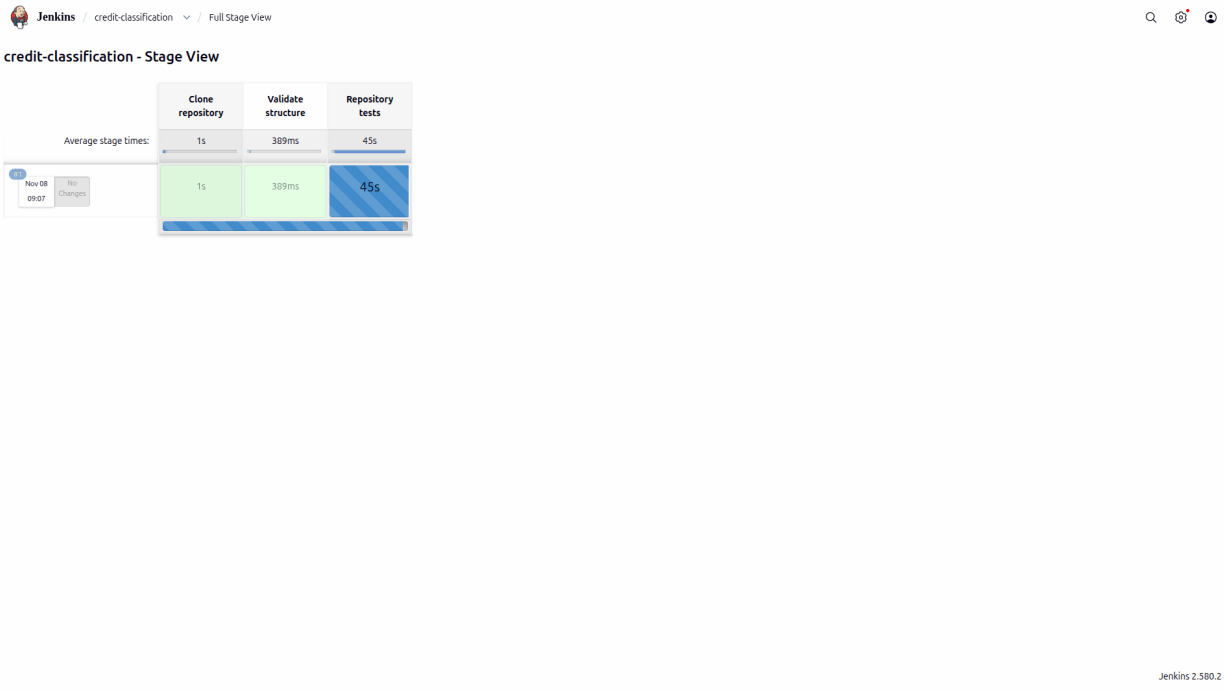



Figura 16

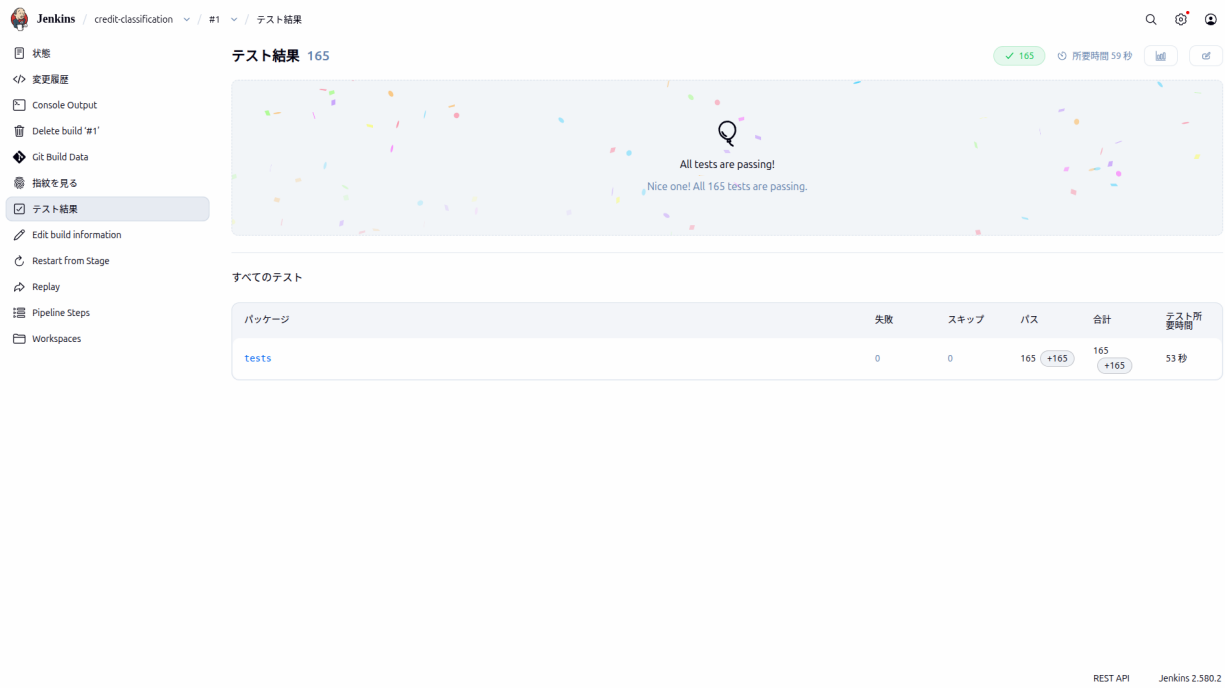



Figura 17

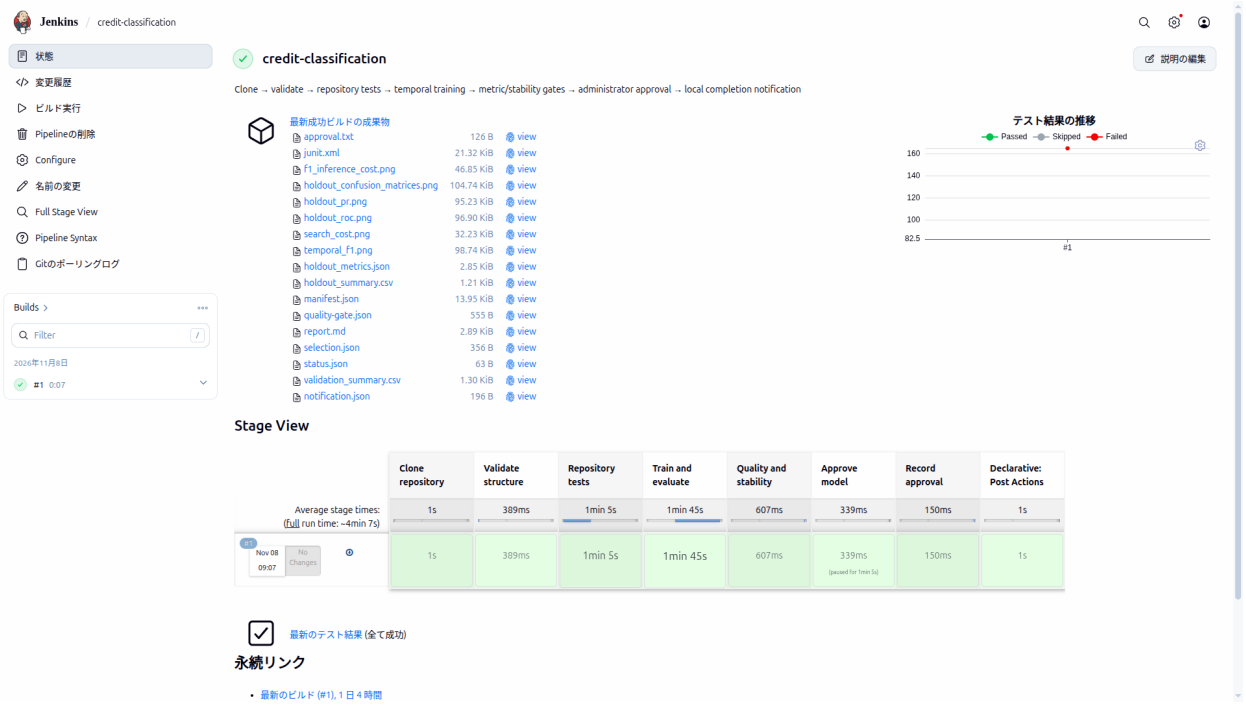



Figura 18

## Jenkins completion receipts

Private local receiver. Times are UTC. No external message service is used.


```
{"job": "credit-classification", "build": "1", "status": "SUCCESS", "url": "http://127.0.0.1:18083/job/credit-classification/1/", "commit":
"242e2edf71981a4ea10dd8bb8756d0b614c569b6", "received_at": "2026-11-08T00:11:16.591332+00:00"}
```


Figura 19

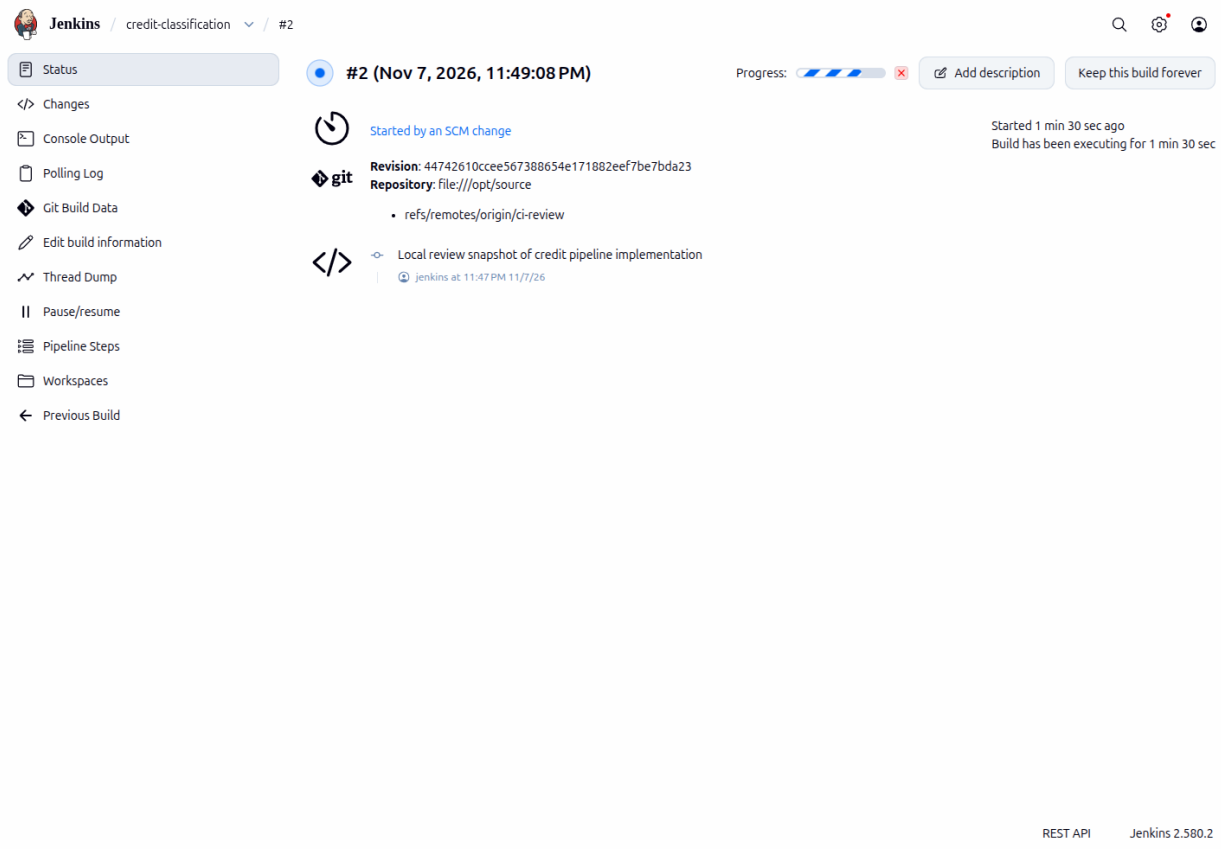



Figura 20

## 2. *Resultados del modelo desarrollado en Python*

Se utilizaron 10.763 registros para evaluar los modelos: 7.534 para entrenamiento y 3.229 para la prueba cronológica. La clase de incumplimiento corresponde a `Pago_atiempo=0`. El conjunto de prueba contiene 108 incumplimientos y 3.121 pagos a tiempo. El modelo se seleccionó con cinco bloques temporales y las semillas 42, 43 y 44, antes de revisar TEST.

| Modelo Python | Accuracy | F1 impago | Recall impago | ROC-AUC | AP |
|---|---|---|---|---|---|
| Regresión logística | 0,9080 | 0,1134 | 0,1759 | 0,6278 | 0,0652 |
| Árbol de decisión | 0,9297 | 0,1498 | 0,1852 | 0,6084 | 0,0591 |
| Random forest seleccionado | 0,8250 | 0,1130 | 0,3333 | 0,6573 | 0,0994 |
| Referencia heurística | 0,7928 | 0,0798 | 0,2685 | 0,6241 | 0,0858 |
| Dummy mayoritario | 0,9666 | 0,0000 | 0,0000 | 0,5000 | 0,0334 |

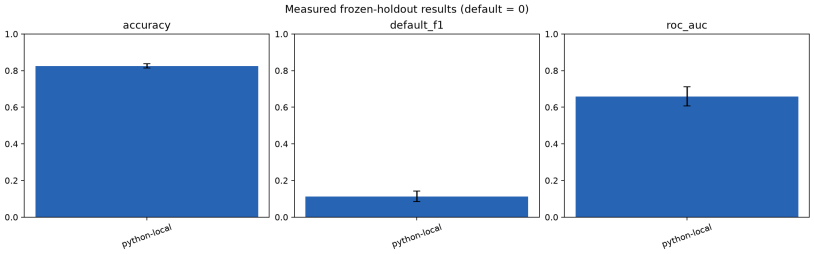



Figura 21

Como se observa en la tabla, el árbol de decisión obtuvo el mayor F1 de incumplimiento en TEST, con un valor de 0,1498. El modelo seleccionado durante la validación fue el random forest. Se conservó esa selección para evitar cambiar de modelo después de conocer los resultados de prueba.

La mayor exactitud (accuracy) fue de 0,9666 y correspondió al modelo que siempre predice la clase mayoritaria (dummy). Sin embargo, este modelo no detectó ningún incumplimiento. Por eso, la exactitud por sí sola no describe la capacidad de detectar impagos. El random forest obtuvo 36 TP (verdaderos positivos), 72 FN (falsos negativos), 493 FP (falsos positivos) y 2.628 TN (verdaderos negativos), con un F1 de 0,1130.

## *3. Resultados del entrenamiento en GCP*

Se realizaron dos entrenamientos en GCP: uno con un pipeline personalizado de Vertex AI y otro con AutoML Tabular. En ambos casos se utilizó el mismo conjunto cronológico de prueba para comparar los resultados.

| **Modalidad** | **Resultado de ejecución** | **Presupuesto o recurso utilizado** |
|---|---|---|
| Vertex AI personalizado | SUCCEEDED; tarea de entrenamiento de 212 s | CPU, sin GPU; imagen y protocolo fijados |
| AutoML Tabular | SUCCEEDED; modelo y evaluación registrados | Presupuesto máximo de 1 hora-nodo; early stopping activo |

Los datos se cargaron en el proyecto de entrenamiento de GCP. Se conservó la división cronológica para comparar los modelos sobre las mismas filas de prueba. Las capturas muestran la carga de los archivos y la configuración inicial de la división manual.

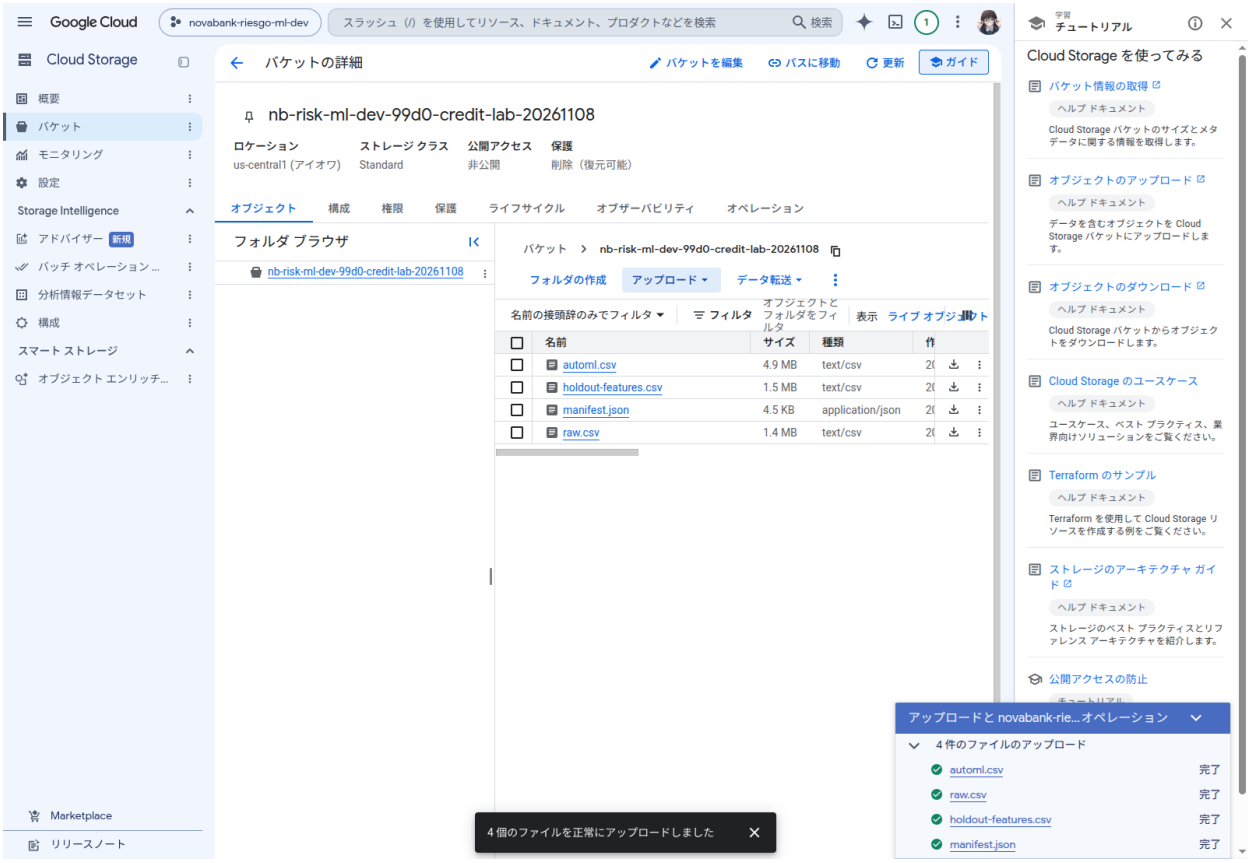

Google Cloud
novabank-riesgo-ml-dev
Cloud Storage
バケットの詳細
バケットを編集
パスに移動
更新
ガイド
nb-risk-ml-dev-99d0-credit-lab-20261108
ロケーション
us-central1 (アイオワ)
ストレージ クラス
Standard
公開アクセス
非公開
保護
削除（復元可能）
オブジェクト
構成
権限
保護
ライフサイクル
オブザーバビリティ
オペレーション
フォルダ ブラウザ
フォルダの作成
アップロード
データ転送
名前
サイズ
種類
automl.csv
4.9 MB
text/csv
holdout-features.csv
1.5 MB
text/csv
manifest.json
4.5 KB
application/json
raw.csv
1.4 MB
text/csv
4 個のファイルを正常にアップロードしました
チュートリアル
Cloud Storage を使ってみる
アップロードと novabank-rie...オペレーション
4 件のファイルのアップロード
automl.csv 完了
raw.csv 完了
holdout-features.csv 完了
manifest.json 完了

Figura 22

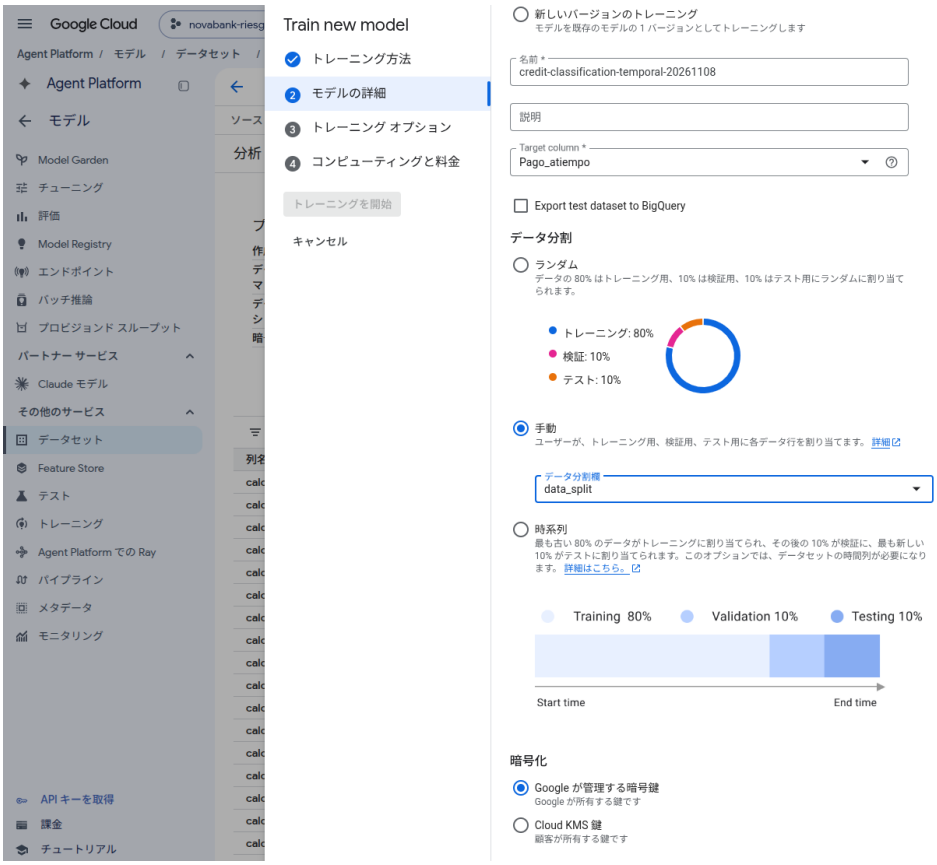



Figura 23

### *3.1. Reproducción del entrenamiento Python en Vertex AI*

El pipeline `credit-cpu-demo-20261108-r3` completó el entrenamiento y guardó el modelo, las métricas y los archivos de evaluación. La tarea utilizó CPU y duró 212 segundos. Las métricas del random forest coincidieron con las obtenidas en Python local, con diferencias inferiores a $10^{-12}$.

Las capturas de seguimiento muestran el DAG del pipeline y la búsqueda de modelos durante la ejecución. La captura de métricas presenta los resultados del entrenamiento personalizado.

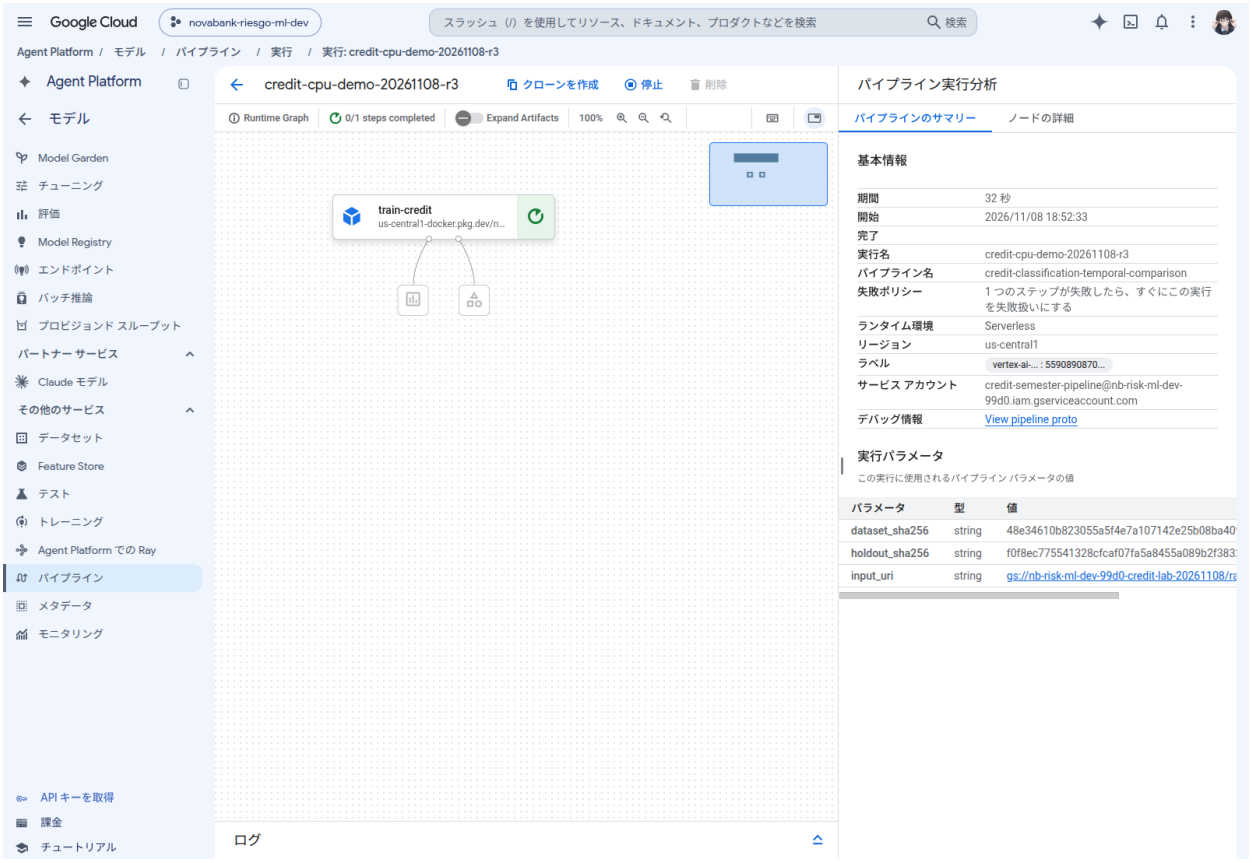

Google Cloud
novabank-riesgo-ml-dev
スラッシュ (/) を使用してリソース、ドキュメント、プロダクトなどを検索
検索
Agent Platform / モデル / パイプライン / 実行 / 実行: credit-cpu-demo-20261108-r3
Agent Platform
モデル
Model Garden
チューニング
評価
Model Registry
エンドポイント
バッチ推論
プロビジョンド スループット
パートナー サービス
Claude モデル
その他のサービス
データセット
Feature Store
テスト
トレーニング
Agent Platform での Ray
パイプライン
メタデータ
モニタリング
API キーを取得
課金
チュートリアル
credit-cpu-demo-20261108-r3
クローンを作成
停止
削除
Runtime Graph
0/1 steps completed
Expand Artifacts
100%
train-credit
us-central1-docker.pkg.dev/n...
ログ
パイプライン実行分析
パイプラインのサマリー
ノードの詳細
基本情報
期間 32 秒
開始 2026/11/08 18:52:33
完了
実行名 credit-cpu-demo-20261108-r3
パイプライン名 credit-classification-temporal-comparison
失敗ポリシー 1 つのステップが失敗したら、すぐにこの実行を失敗扱いにする
ランタイム環境 Serverless
リージョン us-central1
ラベル vertex-ai-... : 5590890870...
サービス アカウント credit-semester-pipeline@nb-risk-ml-dev-99d0.iam.gserviceaccount.com
デバッグ情報 View pipeline proto
実行パラメータ
この実行に使用されるパイプライン パラメータの値
パラメータ 型 値
dataset_sha256 string 48e34610b823055a5f4e7a107142e25b08ba40
holdout_sha256 string f0f8ec775541328cfcaf07fa5a8455a089b2f383
input_uri string gs://nb-risk-ml-dev-99d0-credit-lab-20261108/ra

Figura 24

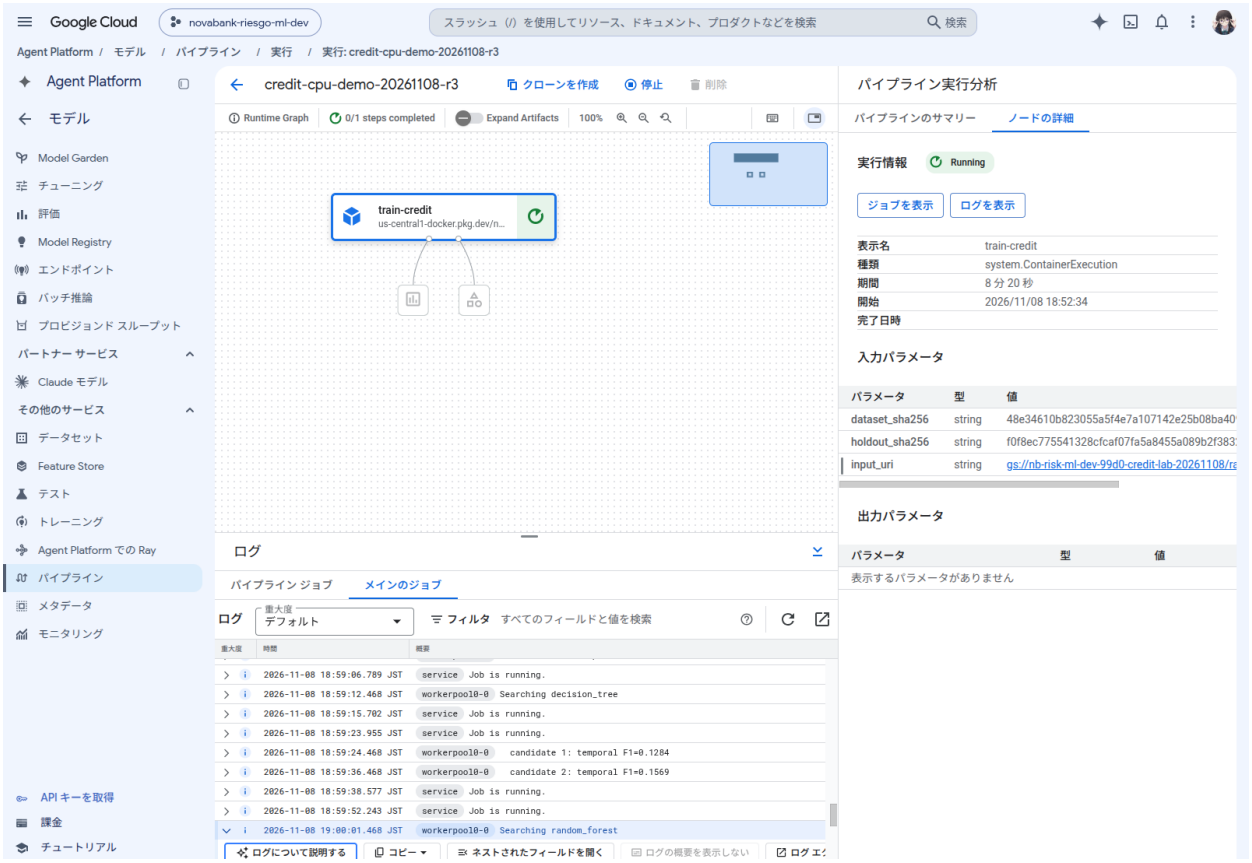



Figura 25

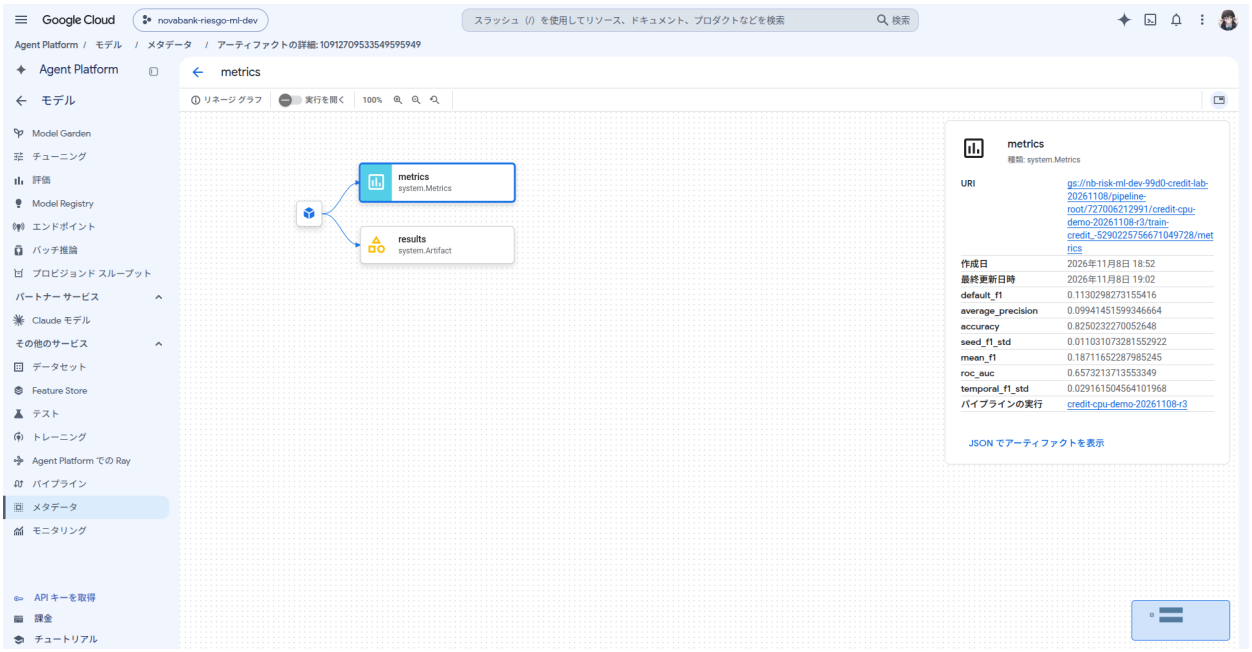



Figura 26

La captura muestra las métricas registradas en Vertex AI. Al utilizar los mismos datos y el mismo procedimiento, se obtuvo el mismo resultado que en Python local. GCP permitió organizar la ejecución y guardar los resultados, pero el cambio de entorno no mejoró por sí mismo el modelo.

### 3.2. *Finalización de AutoML y evaluación de incumplimiento*

El entrenamiento de AutoML finalizó con un presupuesto máximo de 1 hora-nodo, equivalente a 1000 milli-node-hours. El proceso completo duró aproximadamente 2 h 35 min, incluyendo la preparación de los datos y el procesamiento posterior. Ese tiempo total no equivale a 2 h 35 min de horas-nodo facturables.




Figura 27

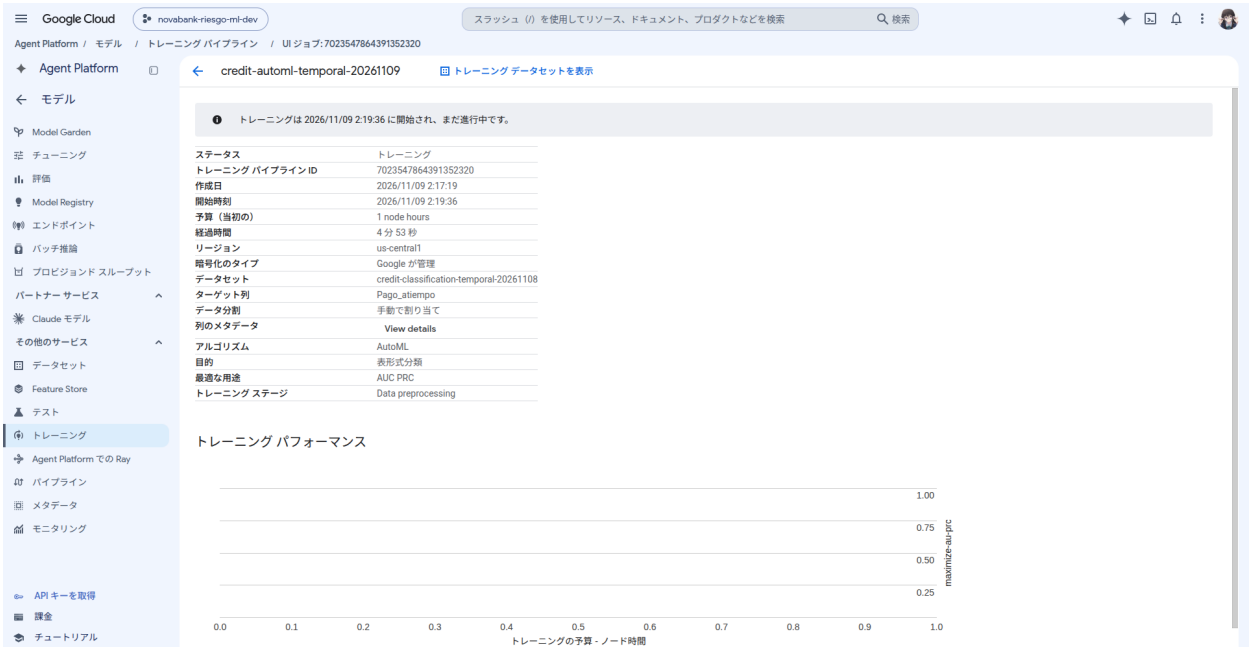



Figura 28

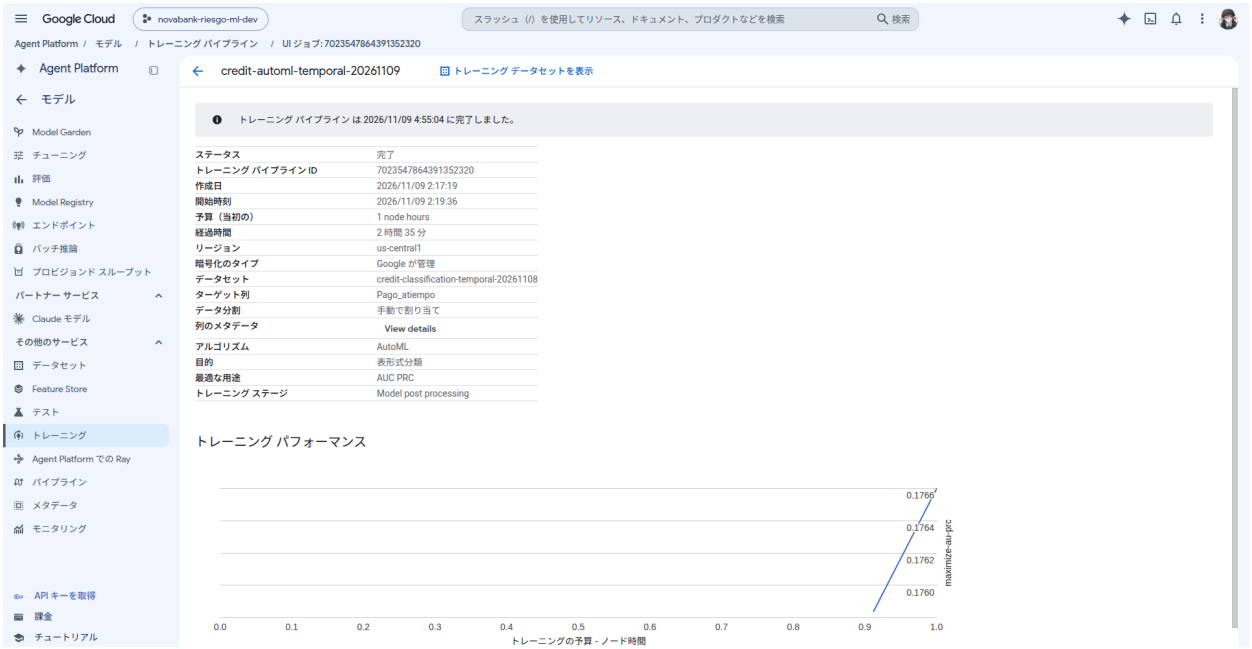



Figura 29

Se mantuvo la división manual de datos: 6.027 registros para entrenamiento (TRAIN), 1.507 para validación (VALIDATE) y 3.229 para prueba (TEST). Para la clase de incumplimiento, AutoML obtuvo un ROC-AUC de 0,6542 y un F1 de 0,1037 con un umbral de 0,5. El modelo quedó registrado, sin desplegar un servicio de predicción.

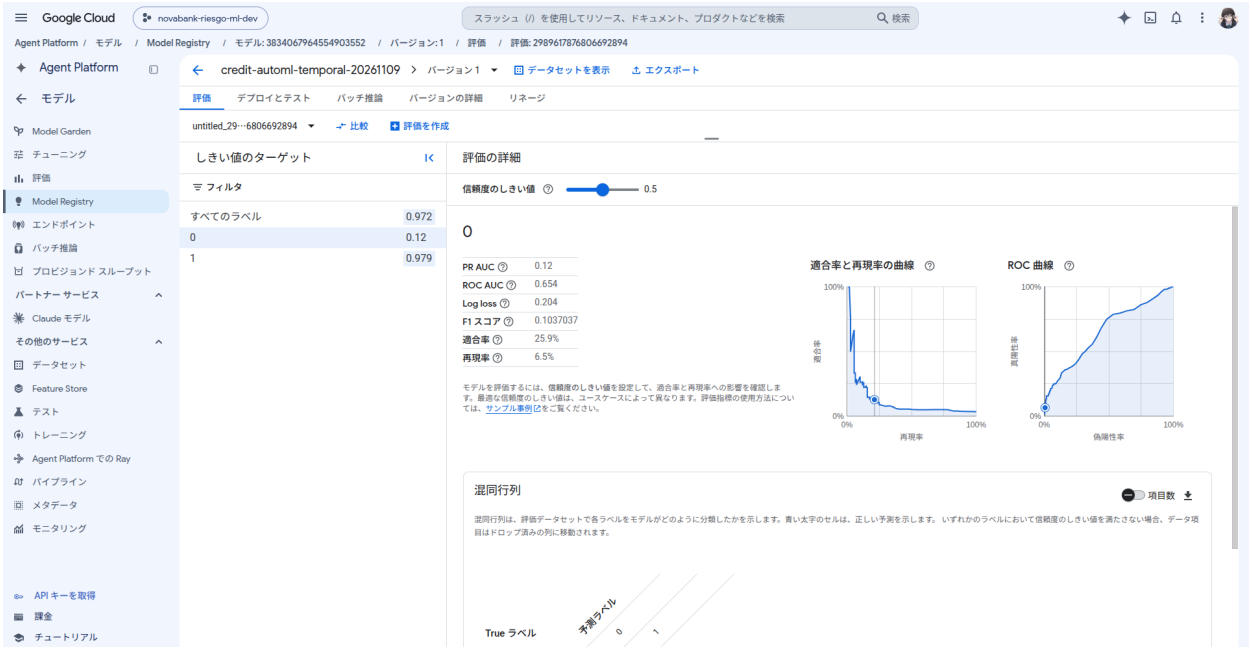



Figura 30

La captura muestra la evaluación de la clase 0, que corresponde al incumplimiento. La evaluación global micro tenía valores más altos, pero incluye la clase mayoritaria. Se utilizó la evaluación de la clase 0 para analizar la detección de impagos.

## *4. Comparación entre Python, Vertex AI y AutoML*

Se compararon los resultados de Python, Vertex AI y AutoML sobre los mismos 3.229 registros de prueba. Python y Vertex AI utilizaron el mismo random forest y obtuvieron las mismas métricas. Para AutoML se tomó la evaluación de la clase 0 que muestra la consola de GCP.

| **Métrica** | **Python** | **Vertex CPU** | **AutoML** |
|---|---|---|---|
| Accuracy | 0,825023 | 0,825023 | 0,962527 |
| Precisión clase 0 | 0,068053 | 0,068053 | 0,259259 |
| Recall clase 0 | 0,333333 | 0,333333 | 0,064815 |
| F1 clase 0 | 0,113030 | 0,113030 | 0,103704 |
| ROC-AUC clase 0 | 0,657321 | 0,657321 | 0,654208 |
| AP sklearn | 0,099415 | 0,099415 | No disponible |
| PR-AUC nativo | No disponible | No disponible | 0,120123 |
| Umbral P(clase 0) | 0,088279 OOF | 0,088279 OOF | 0,5 fijo |

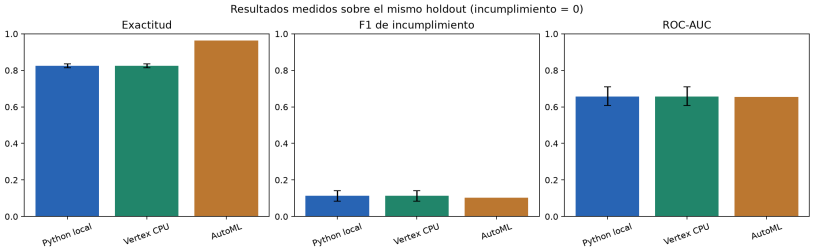



Figura 31

Como se observa en la tabla y en la gráfica, los valores de ROC-AUC fueron cercanos: 0,6573 para Python y Vertex AI, y 0,6542 para AutoML. En esta prueba no se observó una mejora clara con AutoML. No se realizó una prueba estadística para determinar si ambos modelos eran equivalentes.

AutoML obtuvo una exactitud mayor que el random forest, pero menor que el modelo que siempre predice la clase mayoritaria. Para interpretar el resultado también se deben revisar los valores de la matriz de confusión y el umbral utilizado.

No se calculó AP de scikit-learn para AutoML porque no se obtuvieron sus predicciones individuales. La consola presenta un PR-AUC nativo que se calcula de otra manera. Ambos valores se dejaron separados en la tabla, por lo que no se interpreta el valor 0,1201 de PR-AUC como una mejora frente al AP de 0,0994.

### *4.1. Errores observados y efecto del umbral*

Para interpretar la matriz, se considera positiva la clase de incumplimiento (`Pago_atiempo=0`) y negativa la clase de pago a tiempo. Se utilizan TP (verdaderos positivos), TN (verdaderos negativos), FP (falsos positivos) y FN (falsos negativos).

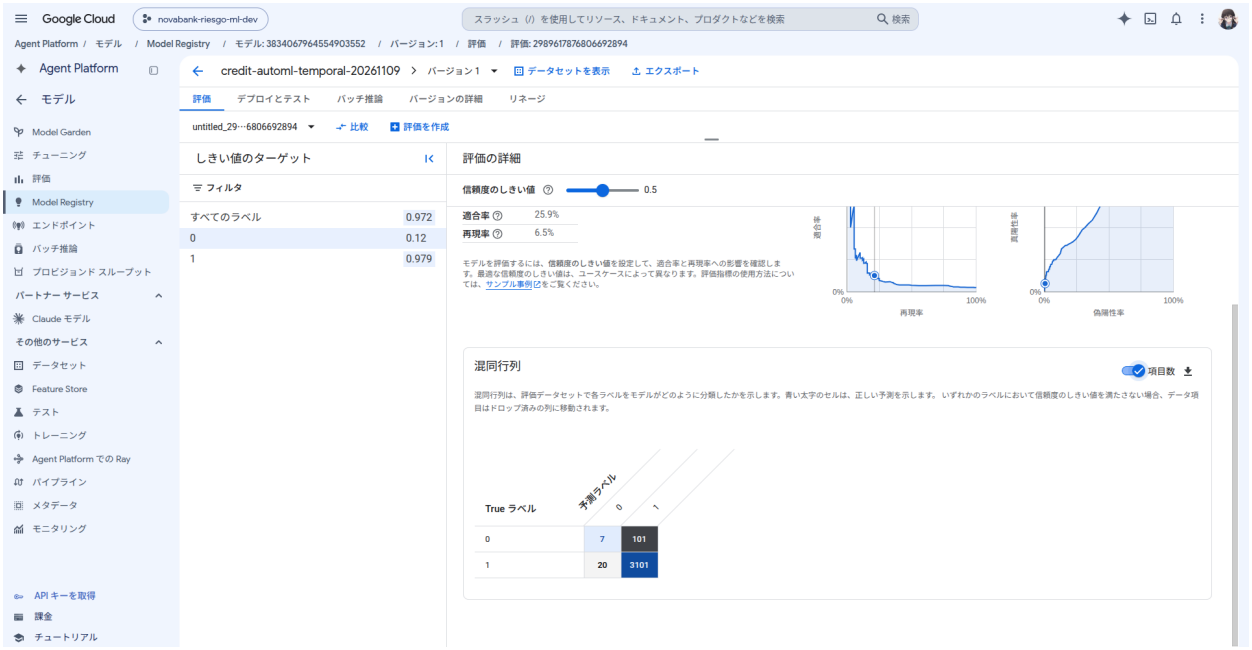



Figura 32

| Resultado en TEST | Python y Vertex CPU | AutoML |
|---|---|---|
| TP (verdaderos positivos) | 36 | 7 |
| TN (verdaderos negativos) | 2.628 | 3.101 |
| FP (falsos positivos) | 493 | 20 |
| FN (falsos negativos) | 72 | 101 |
| Total de incumplimientos (TP + FN) | 108 | 108 |
| Umbral de incumplimiento | Aprox. 0,08828, elegido con validación OOF | 0,5 fijo |

La matriz muestra que AutoML obtuvo menos FP (falsos positivos), pero también menos TP (verdaderos positivos). El random forest registró 36 TP (verdaderos positivos) de los 108 incumplimientos, aproximadamente un tercio, mientras AutoML registró 7 TP (verdaderos positivos), cerca del 6,5 %. Con los umbrales utilizados, AutoML presentó más FN (falsos negativos): 101 frente a 72 del random forest.

Los modelos utilizaron umbrales diferentes y estos no se ajustaron con TEST. Por eso, las diferencias de F1, recall y FP (falsos positivos) dependen tanto del modelo como del umbral.

## *5. Estabilidad, consistencia y límites de la comparación*

El random forest obtuvo un F1 medio de 0,1871 en validación temporal. La desviación fue de 0,0292 entre bloques y de 0,0110 entre semillas, dentro de los límites definidos en Jenkins. En TEST, el F1 bajó a 0,1130. Aunque el modelo cumplió los controles internos, su desempeño fue menor en el período de prueba.

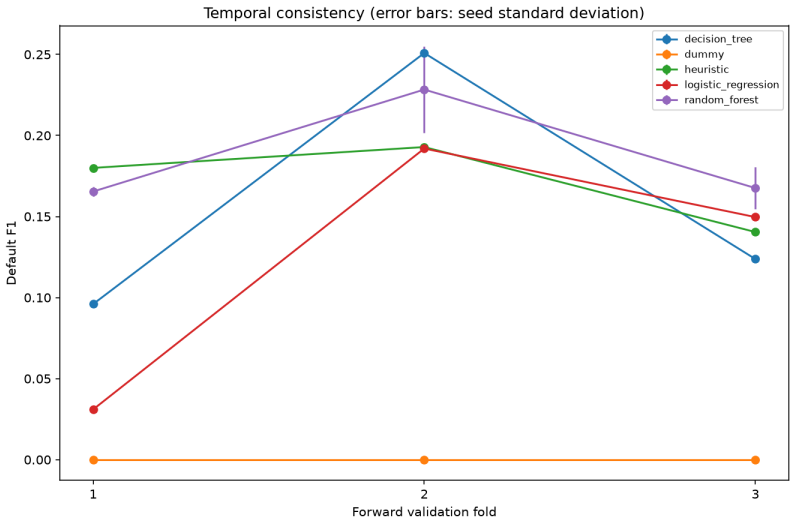



Figura 33

| Indicador del bosque en holdout | Estimación | IC bootstrap 95 % |
|---|---|---|
| Accuracy | 0,8250 | [0,8132; 0,8365] |
| F1 de incumplimiento | 0,1130 | [0,0845; 0,1413] |
| ROC-AUC | 0,6573 | [0,6073; 0,7108] |

Los intervalos de la tabla se calcularon con 200 remuestreos bootstrap estratificados del conjunto de prueba. Estos intervalos describen la incertidumbre de esa muestra y no la variación al entrenar de nuevo el modelo. La proporción de incumplimientos pasó de 5,35 % en entrenamiento a 3,34 % en TEST, por lo que también se deben revisar los cambios entre períodos y la calibración.

| Aspecto | Python / Vertex CPU | AutoML |
|---|---|---|
| Variación temporal F1 | Desviación 0,029162 | No medida en reentrenamientos |
| Variación entre semillas F1 | 0,011031; semillas 42, 43 y 44 | Una ejecución; no medida |
| Incertidumbre en TEST | Bootstrap condicional de predicciones | Sin predicciones individuales; no calculada |
| Consistencia entre entornos | Mismas huellas y métricas <1e-12 | Mismos conteos TEST y cadena de procedencia |

Solo se realizó un entrenamiento de AutoML, por lo que no se pudo medir su estabilidad entre entrenamientos. En Python y Vertex AI se comprobó que los resultados coinciden al utilizar los mismos datos, la misma imagen y el mismo procedimiento. Pero

esta comprobación no permite asegurar el mismo comportamiento en otras poblaciones.

### *5.1. Entrenamiento manual frente a AutoML*

El entrenamiento manual permitió controlar la preparación de las variables, la selección del modelo, los bloques temporales, las semillas, la calibración y el umbral. En Vertex AI se mantuvo este procedimiento y se guardaron la ejecución y sus resultados. Las métricas coincidieron con las obtenidas localmente.

AutoML permitió automatizar parte de la búsqueda del modelo y obtener las métricas y las curvas desde la consola. En este ejercicio utilizó un presupuesto mayor y ofreció menos control sobre la búsqueda interna. No se obtuvo una mejora clara en el ROC-AUC de incumplimiento. Su uso depende del tiempo disponible, del presupuesto y de los controles que requiera la evaluación.

Los tiempos registrados no permiten comparar directamente la velocidad de las dos herramientas. Los 212 segundos corresponden a la tarea personalizada, mientras que el tiempo total de AutoML también incluye la preparación de los datos y otras etapas del servicio.